Experiment-6

Student Name: Ayush Kohli

UID :23BCS11238

Branch: BE-CSE

Section/Group: KRG_3B

Semester: 6th

Date of Performance: 27/02/2026

Subject Name: System Design

Subject Code: 23CSH-314

Aim : To architect a highly scalable and fault-tolerant video streaming system comparable to YouTube, capable of supporting massive video uploads, adaptive bitrate playback, and real-time social interactions while maintaining low latency and high availability.

Objectives:

- To analyze the major architectural building blocks of large-scale video streaming platforms.
- To identify key functional capabilities such as direct-to-cloud uploads and personalized content delivery.
- To establish critical non-functional requirements including CDN utilization, performance optimization, and latency reduction
- To explore asynchronous and event-driven system design to decouple heavy background processing from user-facing APIs.
- To design REST-based service interfaces for managing video lifecycle, discovery, and user engagement activities.

Procedure

- Studied and implemented the Pre-Signed URL upload mechanism to bypass backend bottlenecks during large media transfers.
- Defined both functional specifications and performance expectations required for scalable content distribution.
- Designed RESTful APIs for upload coordination, metadata retrieval, search operations, and user interactions.
- Evaluated adaptive streaming standards such as HLS and DASH, and optimized storage strategies by separating thumbnails from video assets.
- Explored real-time data processing frameworks (Apache Flink integrated with Kafka) for scalable and accurate view-count aggregation

Functional Requirements

- The system must support user onboarding, secure authentication, and profile management.
- The platform must efficiently handle large-scale raw video uploads.
- The system should provide adaptive bitrate streaming based on dynamic network conditions.
- Users must be able to interact through likes, dislikes, and comments in real time.
- The platform must enable video search functionality using titles and metadata.
- A personalized homepage feed should be generated based on user viewing behavior.

Core Entities of the System

- Users
- Videos
- Interactions (Likes/Dislikes)
- Comments
- ViewHistory

API Design

1. Authentication Services

- **POST** /api/v1/auth/register – User registration
- **POST** /api/v1/auth/login – User login

2. Videos Feed Services

- **POST** /api/v1/videos/upload/init – Initialize upload (returns pre-signed cloud storage URL)
- **GET** /api/v1/videos/{video_id} – Retrieve video metadata
- **GET** /api/v1/feed – Fetch personalized feed (supports pagination)
- **GET** /api/v1/search?q={query} – Search videos

3. Interaction Services

- **POST** /api/v1/videos/{video_id}/like – Like a video
- **POST** /api/v1/videos/{video_id}/comments – Add a comment
- **GET** /api/v1/videos/{video_id}/comments – Retrieve comments

Non-Functional Requirements

- The system must support extremely high concurrency for uploads and millions of simultaneous video streams.
- The architecture should eliminate single points of failure to ensure continuous availability.
- Engagement metrics such as views and likes should follow an eventual consistency model to reduce database load.
- Latency must be minimized using geographically distributed CDN edge nodes.
- The transcoding pipeline should scale horizontally to handle sudden spikes caused by viral content.

High-Level Design (HLD)

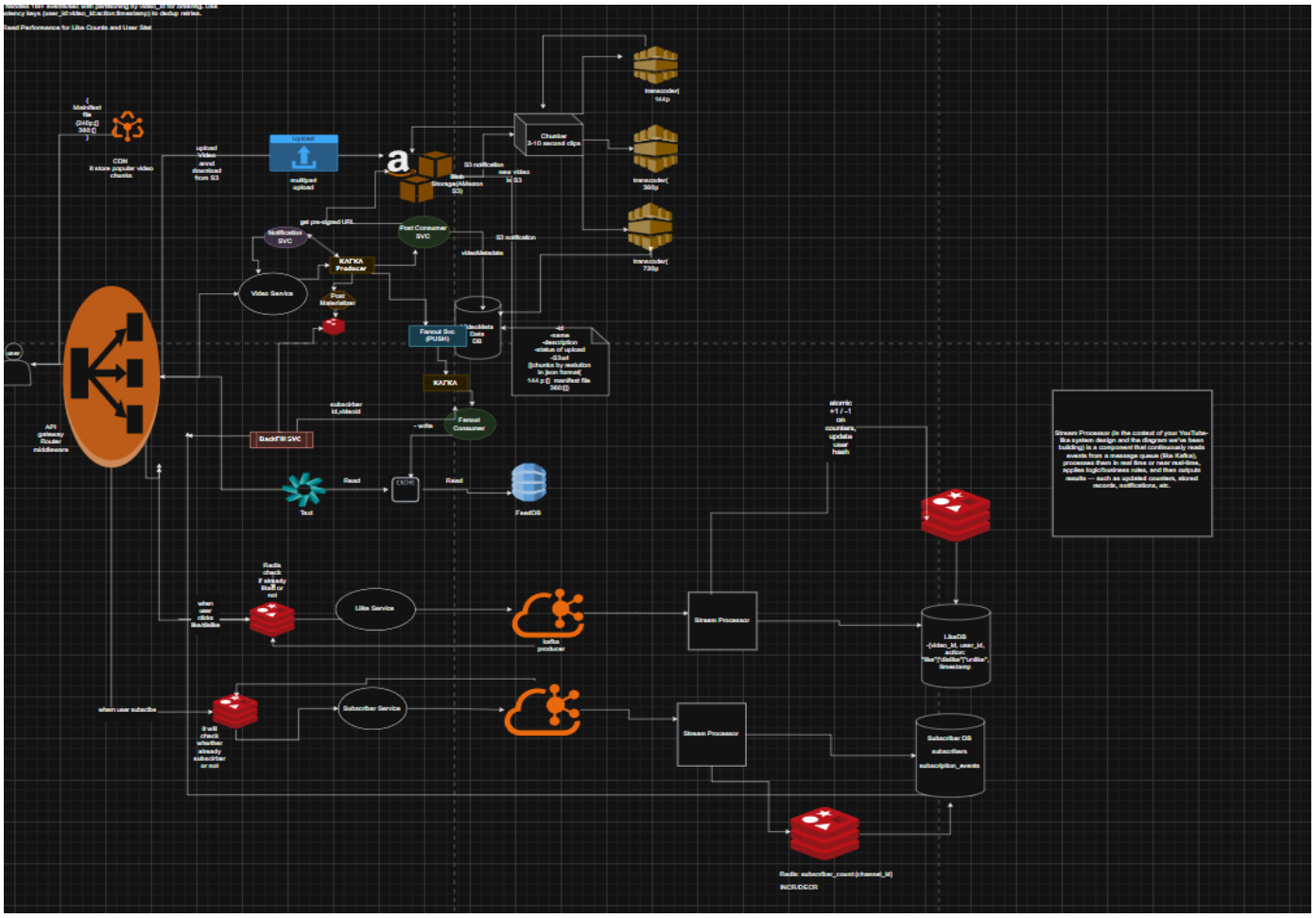
The system architecture consists of client applications interacting with an API Gateway that routes requests to independent microservices, including Authentication, Upload Management, Metadata, Feed Generation, and Interaction services.

Large video files are uploaded directly to cloud object storage through pre-signed URLs, preventing API server overload.

An asynchronous processing pipeline handles video transcoding into adaptive streaming formats (HLS/DASH).

Processed segments are distributed globally through CDN networks for optimized delivery.

High-frequency engagement data is optimized using in-memory caching systems such as Redis for like tracking, NoSQL databases for scalable comment storage, and a Kafka-based streaming pipeline integrated with Apache Flink for real-time and fault-tolerant view count aggregation.



Outcome

- Designed a scalable and resilient event-driven architecture for large-scale video streaming systems.
- Clearly defined both operational features and performance-driven system requirements.
- Developed RESTful APIs integrated with asynchronous background workflows.
- Gained practical understanding of CDN-based distribution, adaptive bitrate streaming, and distributed stream processing for high-traffic applications.